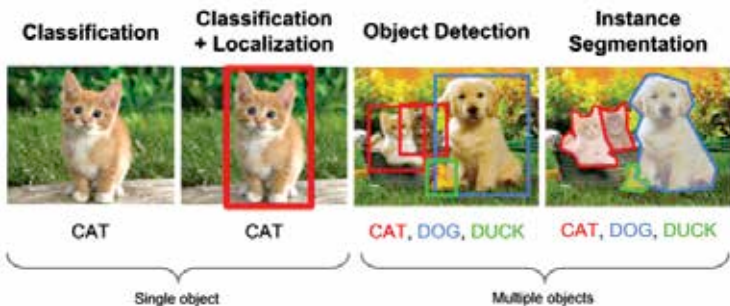


```
eachObject["percentage_probability"] )
• print("-----")
• \\A modified image file named image_new.png will be saved
  with a bounded box.
• from IPython.display import Image
• Image("image_new.png")
• TEST_IMAGE_PATHS = [ os.path.join(PATH_TO_TEST_
  IMAGES_DIR, 'image{}.jpg'.format(i)) for i in range(1, N+1) ]
• write_saved_model(saved_model_path, trained_checkpoint_prefix,
•
• placeholder_tensor, outputs)
```

The code above will first save object references by placing a modified box around all identified objects and then saves them in a predestined folder.

The TEST IMAGE PATHS code will check for the reference points of all the images and compare them with their positions and sizes in the input nodes, thus yielding a graphical result of how close the images are to the actual one.

These results should be further saved for reference in the future by letting the system know about where to write and commit the code with



How Object Detection Works

the output. The same accuracies can be tested on a typical Intel setup that leverages on scalability and performance. Getting an Intel® Dev Cloud account is required and should be done to allow for the system to use the Xeon™ chip's amazing capabilities.